

Laborator 10

Implementarea sistemelor în timp real Partajarea resurselor hardware. Utilizarea semafoarelor

Obiectiv Laboratorul își propune să ilustreze, printr-un exemplu, modalitatea în care se pot partaja între mai multe task-uri resursele hardware de care dispune un sistem în timp real.

Breviar teoretic Așa cum s-a prezentat la curs, alături de resursele hardware alocate la un sistem în timp real pentru un task, pentru nucleul în timp real și pentru sistemul de task-uri este posibil ca aplicația în sine să-și propună, prin task-urile sale accesarea unei-unor anumite resurse hardware unice.

Majoritatea resurselor hardware sunt unice și unic accesibile. Cu alte cuvinte, dacă mai multe task-uri cer controlul unei resurse hardware numai un singur task poate fi servit la un moment dat. Un exemplu de resursă hardware aflată în această situație poate fi însuși procesorul. În cazul acestuia, mecanismul de comutare între task-uri permite ca la un moment dat să fie disponibil numai un singur task pentru execuție. În cazul celorlalte resurse hardware se folosește un mecanism bazat pe semafoare care va permite ca resursa hardware să fie disponibilă, la un moment dat, numai pentru un task.

Un semafor conține:

- numele acestuia, corelat de regulă cu resursa hardware care o administrează;
- valoarea acestuia sau starea acestuia care arată gradul ei de ocupare;
- o listă de așteptare cu task-urile (sau pointeri către acestea) care solicită resursa hardware respectivă dar care, datorită faptului că au găsit-o deja ocupată nu au putut-o accesa și au fost suspendate.

Fiecare resursă hardware are asociată un semafor, care indică gradul de ocupare al acesteia. În cazul în care un task solicită controlul asupra unei resurse hardware dar semaforul asociat acesteia indică faptul că resursa este deja utilizată de către alt task, task-ul curent este eventual suspendat, pus în coada de așteptare și realizată comutarea pe alt task. Prin această procedură se va favoriza executarea task-ului care ocupă resursa, până când acesta o eliberează.

Tratarea task-urilor care au solicitat resursele hardware se face independent de gradul lor de prioritate. Aceasta deoarece, de cele mai multe ori unei resurse hardware externe nu i se poate salva contextul, așa cum se întâmplă în cazul procesorului. De exemplu, dacă un task cu gradul de prioritate 2 solicită accesul la un ecran LCD pentru scrierea un mesaj, acesta va trimite adresa ecranului, adică coordonatele de la care se afișează mesajul, și apoi va trimite șirul. Ecranul, prin logica sa internă, efectuează la fiecare caracter primit pentru afișare o incrementare a adresei pe linie, astfel încât următorul caracter recepționat să fie afișat imediat în stânga celui precedent.

Să ne imaginăm acum că apare un task cu gradul de prioritate 1 care solicită și el dreptul de acces la ecran. În acest caz, nu este posibilă salvarea contextului, respectiv a poziției ecranului, deoarece nu avem acces la ea. Prin urmare, chiar dacă task-ul al doilea rulat are grad de prioritate superior primului, acesta nu poate prelua controlul până când primul task nu își termină sesiunea de lucru cu dispozitivul periferic.

Desfășurarea lucrării În cele ce urmează se va prezenta un exemplu de partajarea unei resurse hardware, este vorba de portul serial pentru transmisie, între două task-uri fără a realiza însă comutarea și suspendarea unuia, așa cum a fost prezentat la curs și în breviarul teoretic de mai sus. Implementarea unui mecanism bazat pe suspendarea task-urilor precum și salvarea acestora în coada de așteptare va reveni în sarcina programatorului, așa cum este precizat în problemele de la sfârșit. Suspendarea aici se realizează prin „blocarea” unui task într-o buclă din care nu iese decât atunci când celălalt task a eliberat resursa hardware.

Exemplu este un program „joc” pentru descoperirea unui cod din 4 cifre. La el sunt necesari 2 jucători. Un jucător introduce un cod din 4 cifre, iar cel de-al doilea jucător va trebui să îl descopere prin încercări. La fiecare încercare se vor afișa câte cifre din codul de încercare sunt egale cu cele din codul care trebuie aflat și se află și pe aceeași poziție, și câte cifre din codul de încercare sunt egale cu cele din codul care trebuie aflat, chiar dacă nu sunt pe aceeași poziție.

De exemplu: dacă codul introdus este 1234 iar un cod încercare este 1111 vor fi 1 situație de egalitate și care ocupă aceeași poziție – cifra 1 de pe prima poziție – și 4 situații în care cifra este prezentă dar nu neapărat pe aceleași poziții – este cazul cifrei 1.

Fiecare încercare este taxată cu 1 punct penalizare. Scopul este acela ca jucătorul să găsească soluția cu cât mai puține puncte de penalizare.

Programul are practic două moduri de lucru: un mod care constă în introducerea codului care trebuie aflat, și un al doilea mod în care al doilea jucător introduce încercările.

Programul va rula pe macheta cu microcontroler C8051F120 din cadrul laboratorului iar comenzile și afișarea datelor se vor face de la PC prin intermediul aplicației HyperTerminal pe portul serial (19200, 8, 1, N).

Programul a fost conceput astfel:

- mesajele citite de la tastatura calculatorului sunt preluate în întreruperea serială ca evenimente spontane, semnalizate prin variabila semafor *sem_receptie* și stocate în variabila *receptie*;
- un task se ocupă de tratarea mesajelor recepționate și semnalizate ca evenimente. Acesta tratează cele două moduri de lucru și afișează caracterele introduse de utilizator prin retransmiterea lor înapoi la consolă;
- un al doilea task se ocupă cu calculul punctajului și transmiterea acestuia, pentru afișare, înapoi la consolă.

Se observă că problema care apare în această problemă este aceea a accesării resursei hard dată de canalul de comunicație serial. Astfel, task-ul care se ocupă de administrarea modurilor de lucru ale programului „joc” solicită canalul serial pentru afișarea caracterelor recepționate și a mesajelor de indicație. Pe de altă parte, task-ul care se ocupă cu calculul punctajului solicită și acesta canalul serial pentru afișarea punctajului. La un moment dat, în timpul transmisiei, comutarea ar duce la o transmitere deficitară. Nici aici nu este posibilă salvarea cumva a contextului perifericului deoarece mesajele o dată transmise nu se poate reveni.

Aici s-a folosit următoarea procedură: task-ul care se ocupă cu modurile de lucru solicită calculul și afișarea din task-ul 2 prin setarea unei variabile semafor denumită

„*sem_calcul*”. După care, task-ul așteaptă până când variabila semafor devine 0, adică atunci când celălalt task a eliberat resursa așa cum se arată mai jos:

```
sem_calcul=1;  
index_cod=0;  
while(sem_calcul);//eliberare resursa seriala si sfarsit calcul  
if(!mod)  
printf("\n\rTastati codul din 4 cifre!\n\r");
```

Task-ul va rămâne blocat în bucla while până când celălalt task a efectuat calculele și a transmis serial rezultatele. Astfel, mesajul informativ care urmează va putea fi transmis dar numai după ce au fost transmise punctele.

Task-ul care se ocupă cu calculul și transmiterea punctelor este descris mai jos:

```
for(;;)  
{  
    if(sem_calcul)  
    {  
        for(i=0;i<4;i++)  
        if(cod[i]==cod_incercare[i])  
        puncte_e++;  
        for(i=0;i<4;i++)  
        for(j=0;j<4;j++)  
        if(cod[i]==cod_incercare[j])  
        puncte_a++;  
        puncte++;  
        printf("\%d\%d\%d\%d\n\r",puncte_e,puncte_a,puncte);  
        if(puncte_e==4)  
        {  
            printf("\n\rCod corect: \%d puncte penalizare\n\r",puncte);  
            mod=0;  
        }  
        sem_calcul=0;  
    }  
}
```

În cele ce urmează se va prezenta listingul complet al programului. Partea de configurare a circuitelor periferice ca acestea să lucreze în modurile de lucru așteptate a fost detaliată la laboratoarele anterioare. Timpul de comutare ales este de 10ms, deci canalul timer va fi configurat pentru a asigura acest timp. Ceasul microsistemului este de 3.0625MHz.

Cod sursă program:

```
#include <c8051f120.h> //fisier pentru declaratii  
#include <stdio.h>  
#define MAX_TASK 2  
//declaratii LED
```

```

sbit LED = P1^6;
sbit TASTA = P3^7;
#define SYSCLK 3062500

//variabile globale unice
unsigned char contor_delay=0;//pentru temporizarea de la ceas
//variabile globale folosite in program de catre nucleul de operare
unsigned char task_crt=0;//task curent
unsigned char task_old=1;//task anterior
unsigned char timp_crt=0;//timpul task-ului curent
unsigned int point;
unsigned char task_varf=0;
//variabile folosite de program
unsigned char cod[4];
unsigned char cod_incercare[4];
unsigned char mod=0,i,j;
unsigned char sem_receptie=0,receptie;
unsigned char index_cod=0;
unsigned int puncte_e,puncte_a,puncte=0;
unsigned char sem_calcul=0;

//declaratia unui task
typedef struct Task
{
    unsigned char stiva_hard[16];//stiva hardware
    //unsigned char stiva_soft[8];//stiva software - set de variabile folosit
    unsigned char pstiva_hard;
    //unsigned char pstiva_soft;
    unsigned char timp;
}task;

//declaratie lista de task-uri
task taskuri[MAX_TASK];

//declaratie tip semafor
typedef struct Semafor
{
    unsigned char stare;
    unsigned char coada[2];
}semafor;

//variabila semafor
semafor semaf;

//task 1
void task1(void)

```

```

{
for(;;)
{
    if(sem_calcul)
    {
        for(i=0;i<4;i++)
            if(cod[i]==cod_incercare[i])
                puncte_e++;
        for(i=0;i<4;i++)
            for(j=0;j<4;j++)
                if(cod[i]==cod_incercare[j])
                    puncte_a++;
        puncte++;
        printf("\%d\%d\%d\n",puncte_e,puncte_a,puncte);
        if(puncte_e==4)
        {
            printf("\nCod corect: %d puncte penalizare\n",puncte);
            mod=0;
        }
        sem_calcul=0;
    }
}
}

```

```

//task 2
void task2(void)
{
for(;;)
{
    puncte=0;
    while(index_cod<4)
    {
        while(!sem_receptie);
        sem_receptie=0;
        cod[index_cod]=receptie;
        index_cod++;
        printf("*");
    }
    index_cod=0;
    printf("\nIntroduceti codul din 4 cifre\n");
    printf("cccc|=~|P\n");

    mod=1;
    while(mod)
    {
        while((index_cod<4) && (mod))

```

```

        {
            while(!sem_receptie);
            sem_receptie=0;
            if(receptie==0x10)
                mod=0;
        else
            {
                cod_incercare[index_cod]=receptie;
                index_cod++;
                printf("%c",receptie);
            }
        }
        puncte_a=0;
        puncte_e=0;
        sem_calcul=1;
        index_cod=0;
        while(sem_calcul);//eliberare resursa seriala si sfarsit calcul
        if(!mod)
            printf("\n\rTastati codul din 4 cifre!\n\r");

    }
}

//functie care adauga un task la o lista de task-uri
void adauga(unsigned char timp,void* functie)
{
    if(task_varf<MAX_TASK)
    {
        point = (unsigned int) functie;//conversie adresa functie
        taskuri[task_varf].stiva_hard[1]=(unsigned char) ((point>>8)&0x00ff);//salvare adresa
        stiva
        taskuri[task_varf].stiva_hard[0]=(unsigned char) (point&0x00ff);
        taskuri[task_varf].pstiva_hard= (taskuri[task_varf].stiva_hard)+5;//salvare stiva
        taskuri[task_varf].timp=timp;
        task_varf++;
    }
}

//functii initializare
//initializare sistem
void Init_System(void)
{
    //dezactivare timer watch-dog
    WDTCN = 0xde;//succesiune de secvente pentru dezactivare watchdog
    WDTCN = 0xad;

```

```
}
```

```
//initializare canale timer 0 si 1
```

```
void Init_TMR(void)
```

```
{
```

```
  SFRPAGE = TIMER01_PAGE;//pregateste pagina pentru configurare timer 0 si 1
```

```
  TCON = 0x50;//activare canal timer 0 si timer 1
```

```
  TMOD = 0x21;//temporizator canal 1 mod 2 (8 biti cu revenire), canal 0 mod 1 (16 biti fara revenire)
```

```
  CKCON = 0x10;//selectie sursa ceas timer 1 - fara prescaler 12
```

```
  TH1 = 0xfb;// - comunicare 19200bps
```

```
  TH0 = 0x9c;//divizare pentru a obtine 100 herti (10 ms) la timer 0
```

```
  TL0 = 0x50;//
```

```
}
```

```
//initializare porturi I/O
```

```
void Init_PORT(void)
```

```
{
```

```
  SFRPAGE = CONFIG_PAGE;//pagina pentru configurare porturi
```

```
  P0MDOUT = 0x03;//configureaza pinul TX ca iesire
```

```
  XBR0 = 0x04; //configureaza portul UART0 pe porturile 0
```

```
  //configurare port 2
```

```
  XBR2 = 0x40;//configureaza liniile de port
```

```
  P1MDOUT |= 0x40;//configureaza pinul P1.6 ca iesire (LED)
```

```
}
```

```
//initializare canal serial
```

```
void Init_UART0(void)
```

```
{
```

```
  SFRPAGE = UART0_PAGE;//setare pagina SFR pentru canalul 0 UART
```

```
  SCON0 = 0x50;//serial modul 1 receptie activata
```

```
  TI0 = 1;//initializare transmisie seriala
```

```
}
```

```
//activare intreruperi
```

```
void enable_interrupt(void)
```

```
{
```

```
  SFRPAGE = LEGACY_PAGE;
```

```
  //activare intrerupere
```

```
  IE = 0x92;
```

```
  //initializare pagina seriala
```

```
  SFRPAGE = UART0_PAGE;
```

```
}
```

```
void main(void)
```

```

{
Init_System();//initializare sistem
Init_TMR();//initializare canale timer
Init_PORT();//initializare porturi I/O
Init_UART0();//initializare canale seriale

//adauga cele doua task-uri la lista de task-uri
adauga(10,task1);
adauga(10,task2);

//initializare stiva pentru task 1
SP = (taskuri[0].stiva_hard)-1;
timp_crt=taskuri[0].timp;

enable_interrupt();//activare intreruperi

//pornire task 1 initial
printf("\r\nTastati codul din 4 cifre!\r\n");

task1();
for(;;);

}

//subrutina tratare timer
void Timer0_ISR (void) interrupt 1
{
TH0 = 0xf6;//divizare cu 250 - comunicare 9600bps
TL0 = 0x08;//divizare pentru a obtine 100 herti (10 ms)

//contor pentru timp
if(contor_delay)
    contor_delay--;

//contor pentru terminarea timpului de executie al unui task curent
if(timp_crt)
    timp_crt--;
else
{
//comutare task-uri
task_old=task_crt;
task_crt++;
if(task_crt==2)

```

```

        task_crt=0;

//comutare stive soft si hard
taskuri[task_old].pstiva_hard=SP;

SP = taskuri[task_crt].pstiva_hard;

}

}

//subrutina tratare intrerupere seriala
void Serial0_ISR (void) interrupt 4
{
//receptie caracter serial?
if(RI0)
{
        sem_receptie=1;
        receptie=SBUF0;
        RI0=0;
}
}

```

Programul se va introduce în mediul de dezvoltare Silicon Laboratories IDE pentru microcontrolerul C8051F120 folosit pe machetele de laborator. Se va compila, construi fișierele obiect, și apoi se va rula programul.

Aplicații și probleme:

1. Se va introduce și rula programul. Se vor observa modurile de lucru.
2. Cifrele care fac parte din cod sunt cuprinse între 0 și 9. Să se introducă o limitare astfel încât acestea să nu poată fi cuprinse decât între 1 și 4.
3. Să se adauge ca parametrul al unui task starea acestuia. Să se definească stările READY și SUSPEND. Toate task-urile care sunt adăugate se vor inițializa în starea SUSPEND.
4. Să se impună condiția ca task-ul următor care va fi comutat să se afle în starea READY. Dacă acesta se află în starea SUSPEND nu va trece ca task curent, ci va lăsa locul task-ului următor.
5. Să se realizeze un sistem care permite implementarea unui semafor cu coadă, așa cum a fost prezentat la curs. Când resursa hardware este ocupată task-ul va fi trecut în starea SUSPEND iar când este eliberată va trece în starea READY.